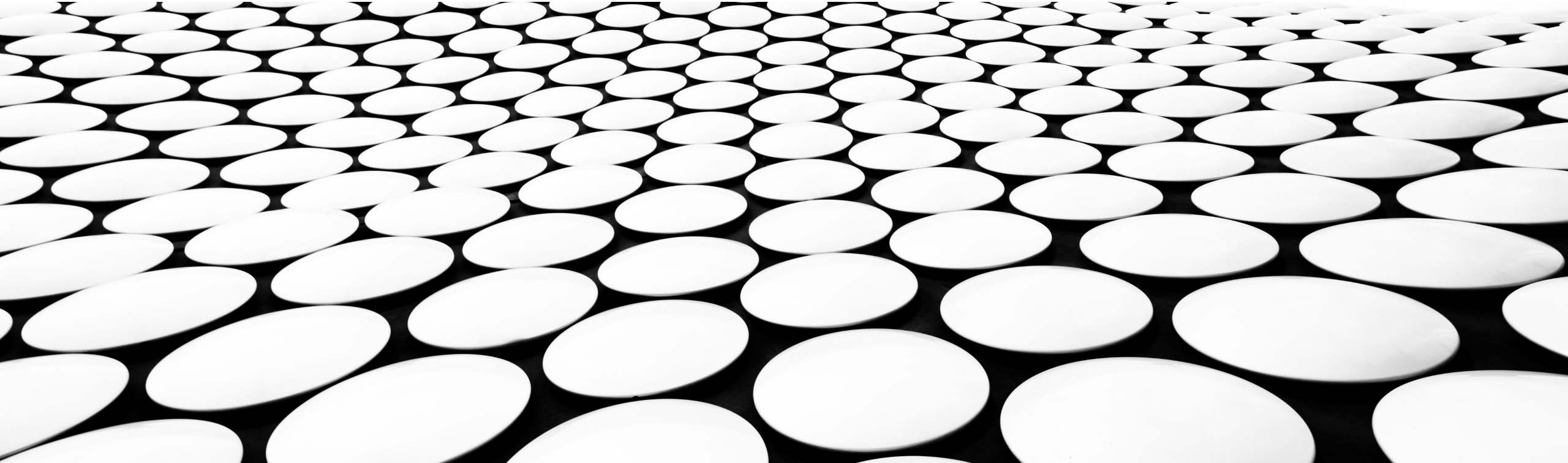
Introduction to Machine Learning

part 3



References

Parts of this lecture are based on:

1. Slides from the Harvard Course CS109A Introduction to Data Science by Pavlos Protopapas, Kevin Rader and Chris Tanner, available at https://github.com/Harvard-IACS/2019-CS109A

2. James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. An introduction to statistical learning. Vol. 112. New York: springer, 2013.

3. Prof. Alex Ihler, CS273a: Introduction to Machine Learning, Ensembles: Bagging, Gradient Boosting, AdaBoost http://sli.ics.uci.edu/Classes/2012F-273a

4. Obiamaka Agbaneje, “Building a Naive Bayes Text Classifier with scikit-learn” @ EuroPython 2018, Edinburgh

Agenda

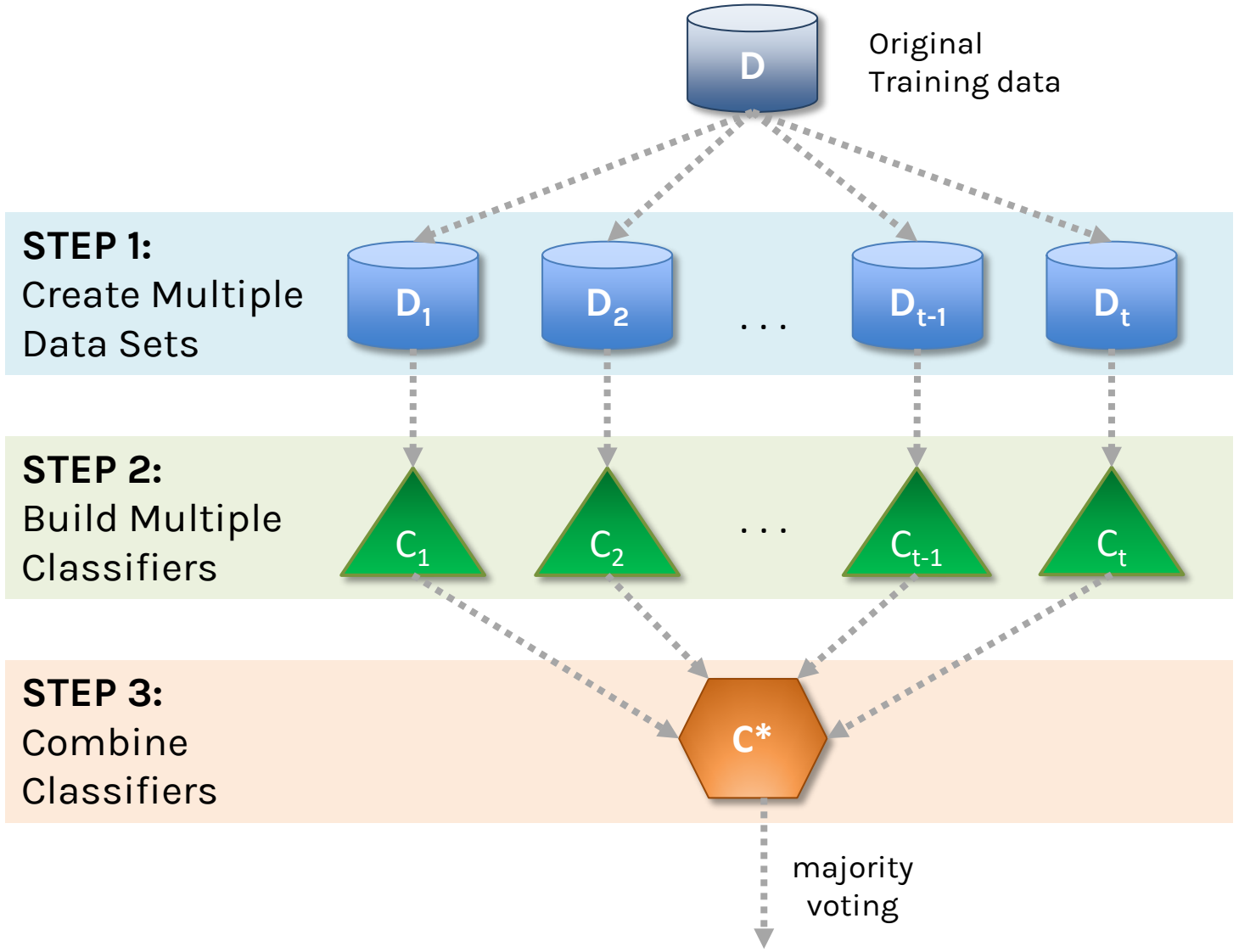
- Ensemble models
- Bagging
- Random Forest
- Boosting
    - XGBoost

- Bayes Classifier

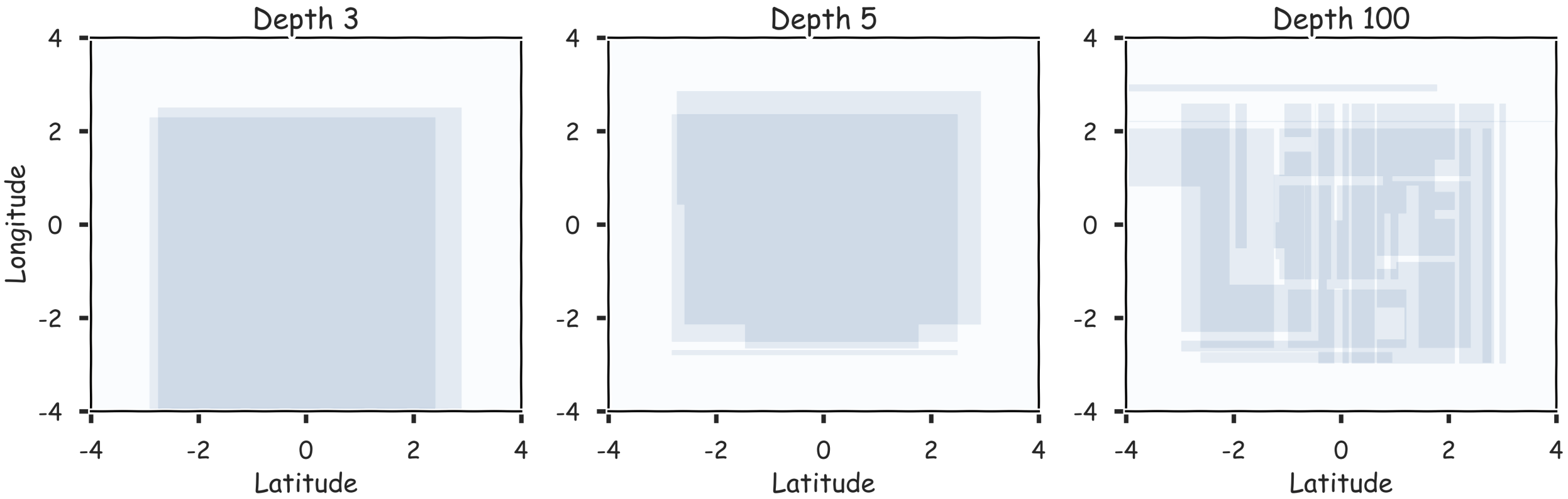
Limitations of Decision Tree Models

- Decision trees models are **highly interpretable** and **fast to train**, using our greedy learning algorithm.
- To **capture a complex decision boundary** (or to approximate a complex function), we need to use a **large tree** (since each time we can only make axis aligned splits).
  - We've seen that large trees have **high variance** and are prone to **overfitting**.
- For these reasons, in practice, decision tree models often **underperforms** when compared with other classification or regression methods.

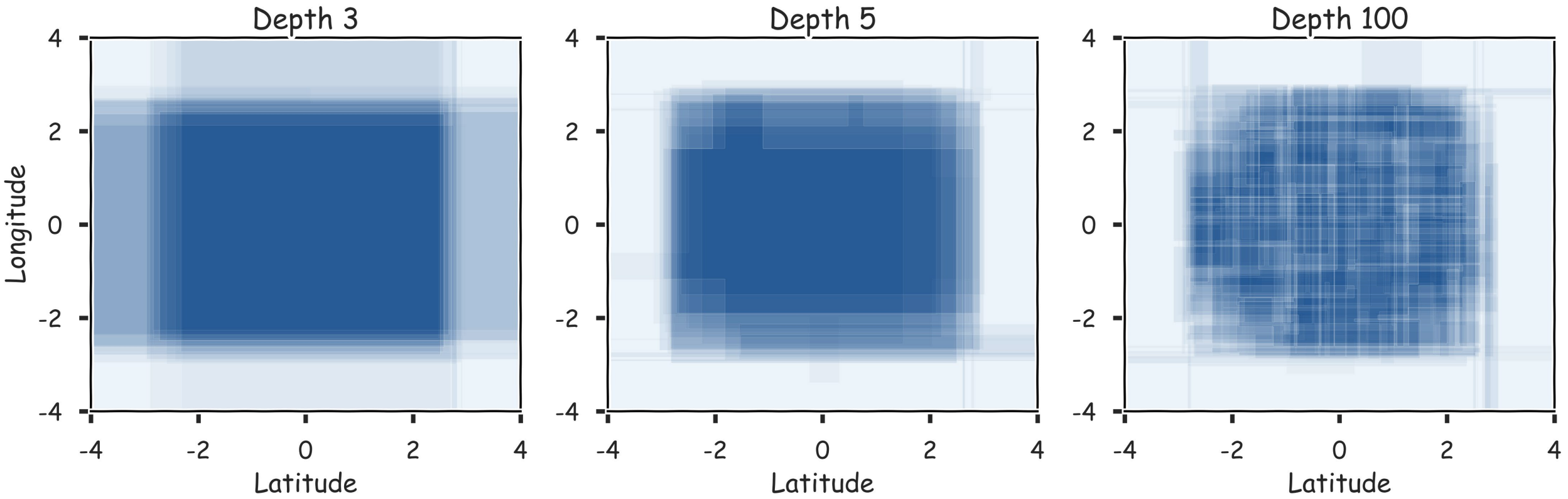
General Idea (Bagging)



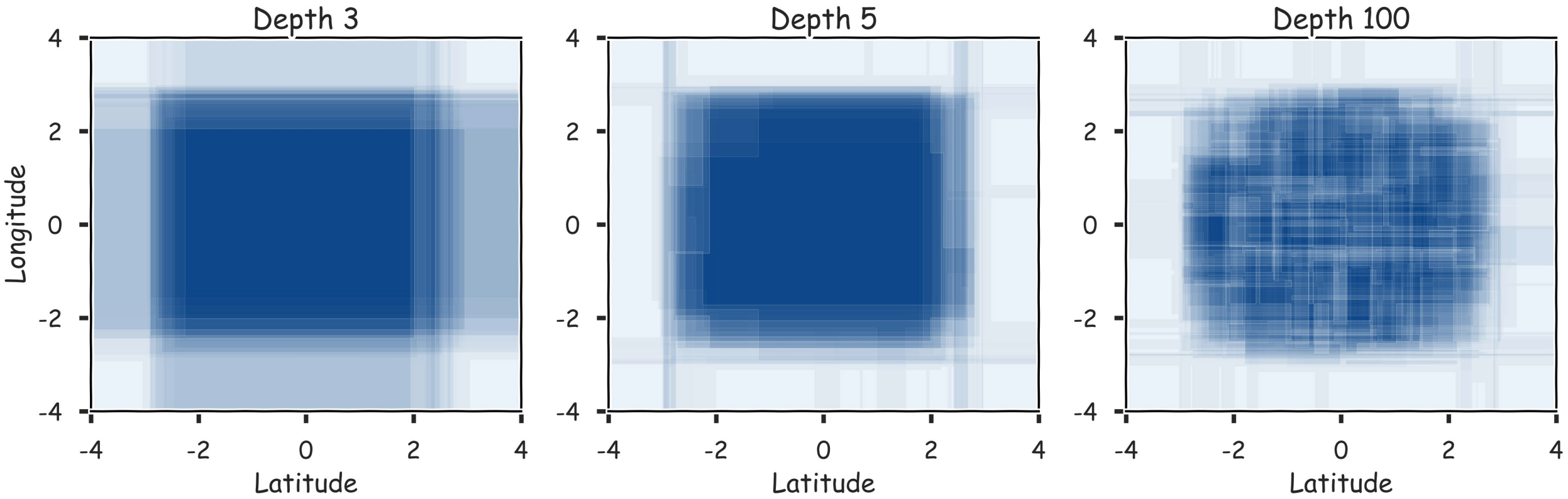
Bootstrap: Combine 2 models



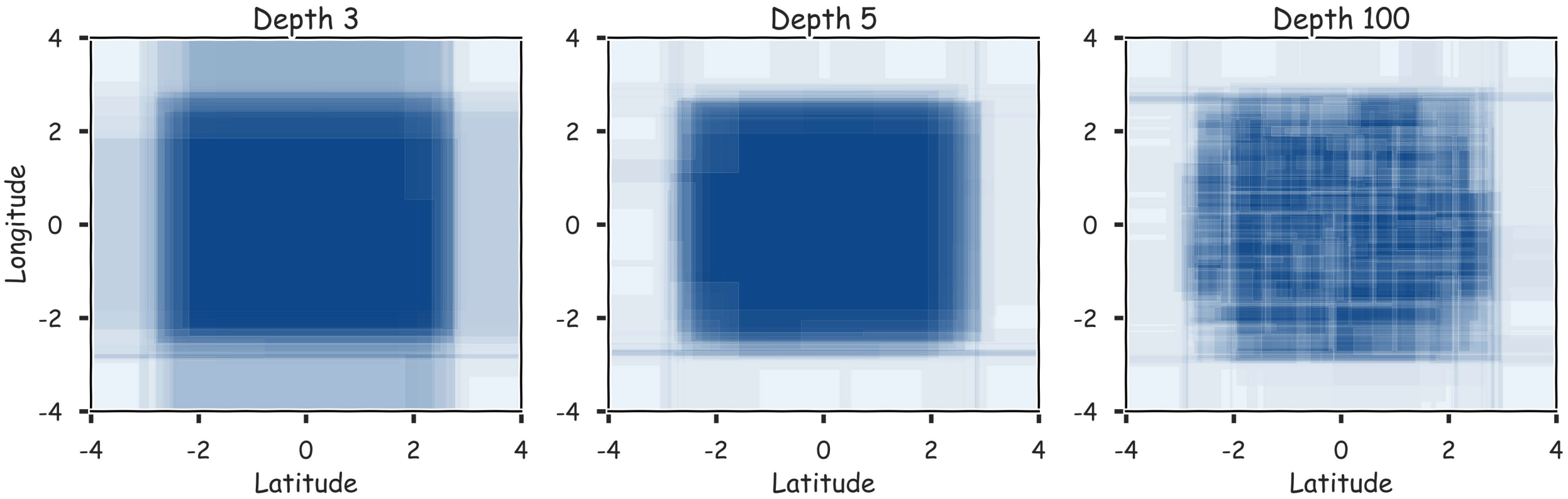
Bootstrap: Combine 20 models



Bootstrap: Combine 100 models



Bootstrap: Combine 300 models



Bagging

- One way to adjust for the high variance of the output of an experiment is to perform the experiment multiple times and then average the results.
- This method is called **Bagging** (Breiman, 1996), short for, of course, Bootstrap Aggregating.
- The same idea can be applied to **high variance** models:
    1. **(Bootstrap)** we generate multiple samples of training data, via bootstrapping. We train a full decision tree on each sample of data.
    2. **(Aggregate)** for a given input, we output the averaged outputs of all the models for that input.
        - For classification, we return the class that is outputted by the plurality of the models.
        - For regression we return the average of the outputs for each tree.

Bagging

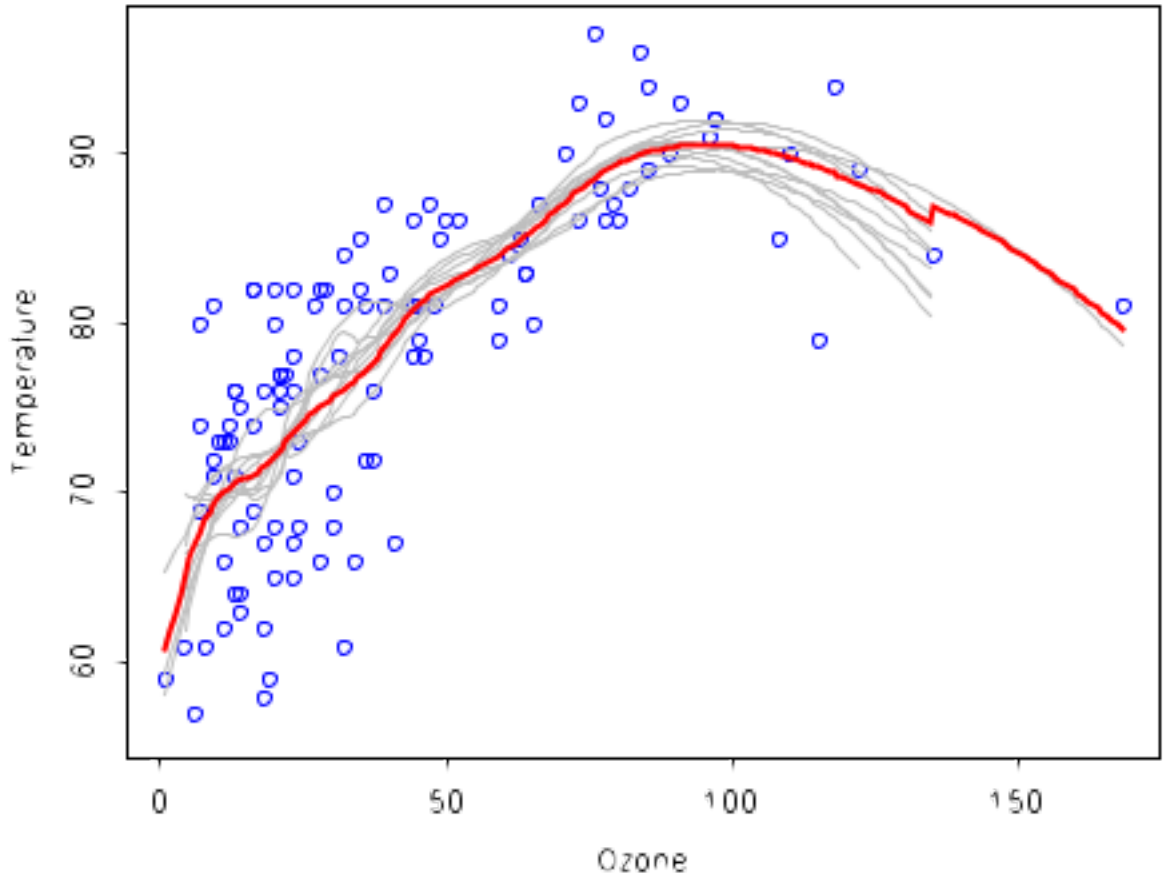
- Note that bagging enjoys the benefits of:

    1. **High expressiveness** - by using full trees each model can approximate complex functions and decision boundaries.

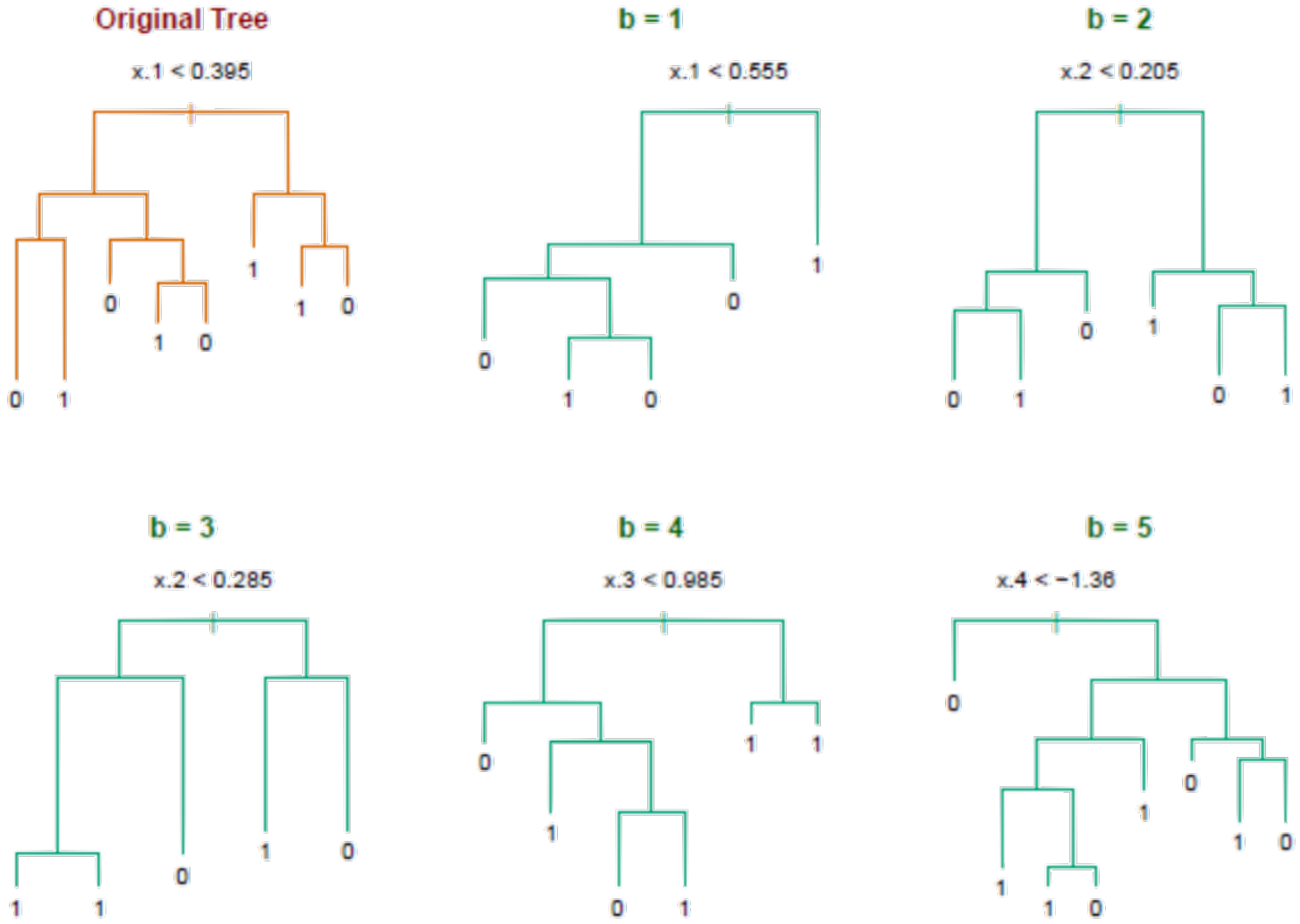
    2. **Low variance** - averaging the prediction of all the models reduces the variance in the final prediction (if we choose a sufficiently large number of trees).

- The **major drawback** of bagging (and other **ensemble methods** that we will study) is that the averaged model is **no longer easily interpretable** - i.e. one can no longer trace the 'logic' of an output through a series of decisions based on predictor values!

Bagging (regression)



Bagging (classification)



Bagging: The process

Original Data

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
⋮	⋮
X_n	y_n

Bootstrap Sample 1

X	Y
X_4	y_4
X_{14}	y_{14}
X_{11}	y_{11}
X_2	y_2
X_{35}	y_{35}
⋮	⋮
X_k	y_k

Decision Tree 1

Original Data

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
⋮	⋮
X_n	y_n

used data
unused data

Bagging: The process

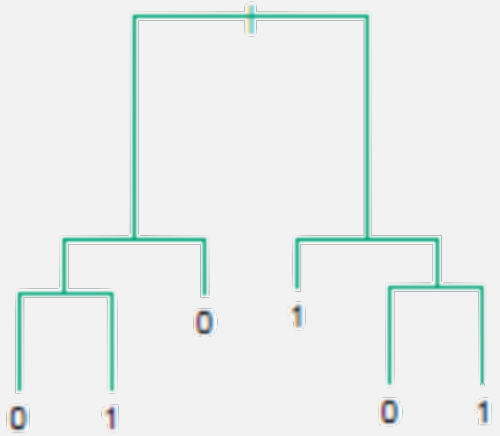
Original Data

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
⋮	⋮
X_n	y_n

Bootstrap Sample 2

X	Y
X_5	y_5
X_3	y_3
X_{12}	y_{12}
X_{43}	y_{43}
X_1	y_1
⋮	⋮
X_k	y_k

Decision Tree 2



Original Data

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
⋮	⋮
X_n	y_n

used data
unused data

Bagging: The process

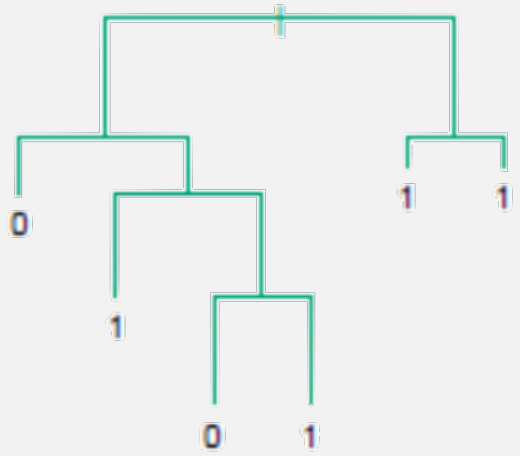
Original Data

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
⋮	⋮
X_n	y_n

Bootstrap Sample 3

X	Y
X_9	y_9
X_4	y_4
X_1	y_1
X_2	y_2
X_{65}	y_{65}
⋮	⋮
X_k	y_k

Decision Tree 3



Original Data

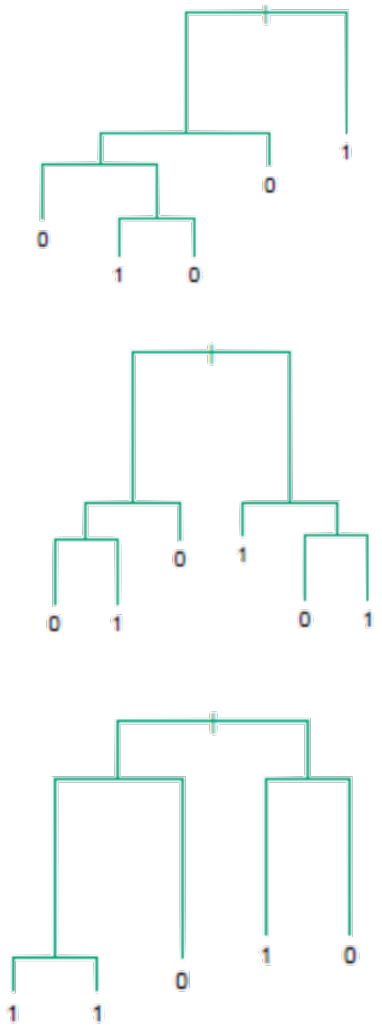
X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
⋮	⋮
X_n	y_n

used data
unused data

Point-wise out-of-bag error

B Trees that did not see $\{X_i, y_i\}$

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
⋮	⋮
X_i	y_i
⋮	⋮
X_n	y_n



CLASSIFICATION

$$\hat{y}_{i,pw} = \text{majority}(\hat{y}_i)$$

prediction

$$e_i = \mathbb{I}(\hat{y}_{i,pw} \neq y_i)$$

error

REGRESSION

$$\hat{y}_{i,pw} = \frac{1}{b}\sum_{j \in B} \hat{y}_{i,j}$$

prediction

$$e_i = \left(y_i - \hat{y}_{i.pw}\right)^2$$

error

Out-of-Bag Error

- Bagging is an example of an **ensemble method**, a method of building a single model by training and aggregating multiple models.
- Given a training set and an ensemble of models, each trained on a bootstrap sample, we compute the **out-of-bag error** of the averaged model by
    1. For each point in the training set, we average the predicted output for this point over the models whose bootstrap training set excludes this point.
    2. We compute the error or squared error of this averaged prediction. Call this the **point-wise out-of-bag error**.
    3. The **out-of-bag error** is obtained by **averaging** the point-wise out-of-bag error over the full training set.

Out-of-Bag Error

- We average the point-wise out-of-bag error over the full training set.

Classification

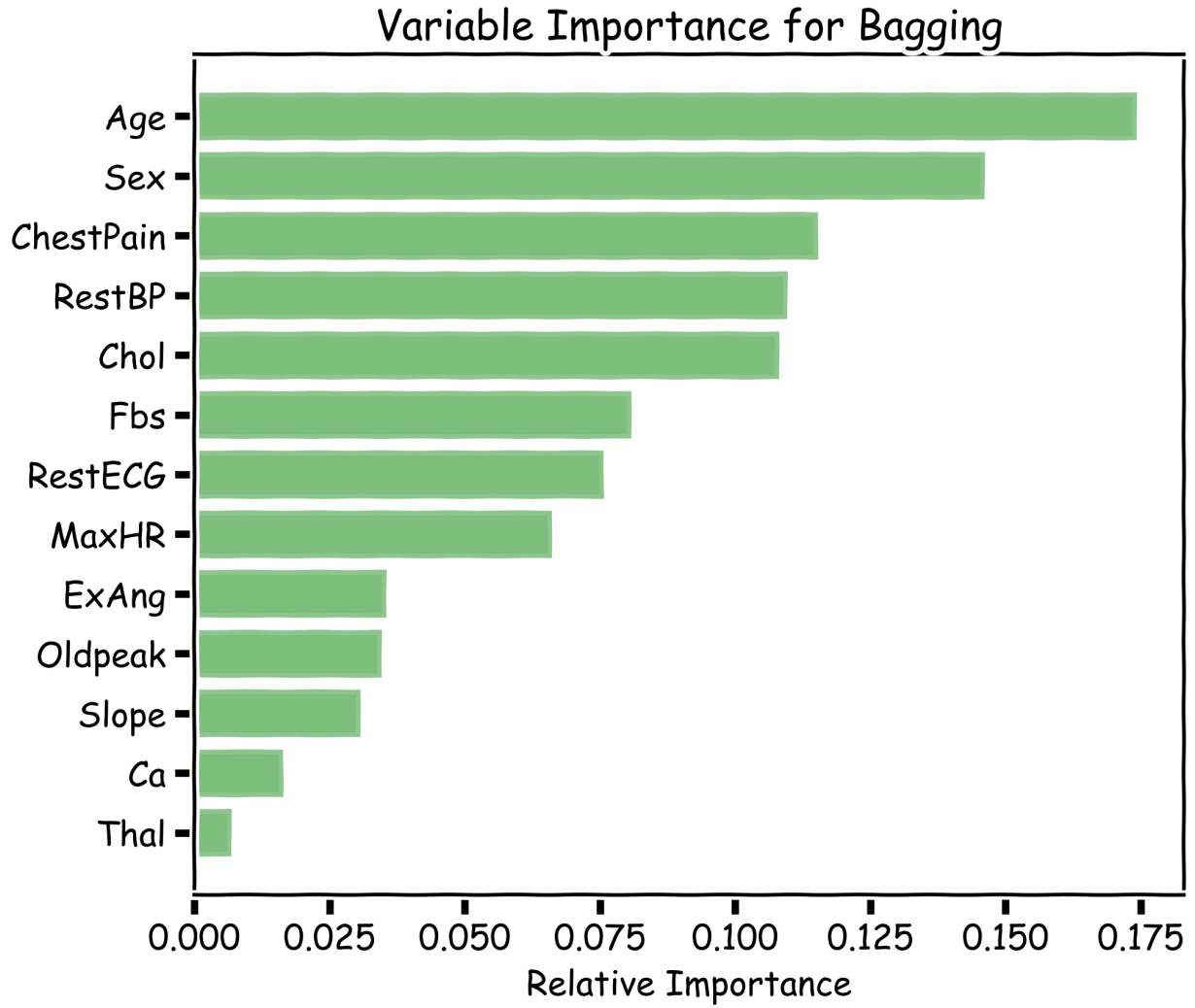
$$Error_{OOB} = \frac{1}{n}\sum_{i}^{n} e_i = \frac{1}{n}\sum_{i}^{n} \mathbb{I}(\hat{y}_{i,pw} \neq \mathrm{y_i})$$

Regression

$$Error_{OOB} = \frac{1}{n}\sum_{i}^{n} e_i = \frac{1}{n}\sum_{i}^{n} \left(y_i - \hat{y}_{i,pw}\right)^2$$

Variable Importance for Bagging

- Bagging improves prediction accuracy at the expense of interpretability.
- Calculate the total amount that the **MSE** (for regression) or **Gini index** (for classification) is decreased due to splits over a given predictor, averaged over all B trees.



100 trees, max_depth=10

Improving on Bagging

- In practice, the ensembles of trees in Bagging tend to be **highly correlated.**

- Suppose we have an **extremely strong predictor,** x_j , in the training set amongst moderate predictors.
  - Then the greedy learning algorithm ensures that most of the models in the ensemble will choose to split on x_j in early iterations.
  - Each tree in the ensemble is identically distributed, with the expected output of the averaged model the same as the expected output of any one of the trees.

Random Forests

- **Random Forest** is a modified form of bagging that creates ensembles of independent decision trees.

- To de-correlate the trees, we:

    1. train each tree on a separate bootstrap sample of the full training set (same as in bagging)

    2. for each tree, at each split, we **randomly** select a set of J' predictors from the full set of predictors.

    3. From amongst the J' predictors, we select the optimal predictor and the optimal corresponding threshold for the split.

Tuning Random Forests

- Random forest models have multiple hyper-parameters to tune:

    1. the **number of predictors** to randomly select at each split

    2. the **total number of trees** in the ensemble

    3. the **minimum leaf node size**

- In theory, each tree in the random forest is full, but in practice this can be computationally expensive (and added redundancies in the model), thus, imposing a minimum node size is not unusual.

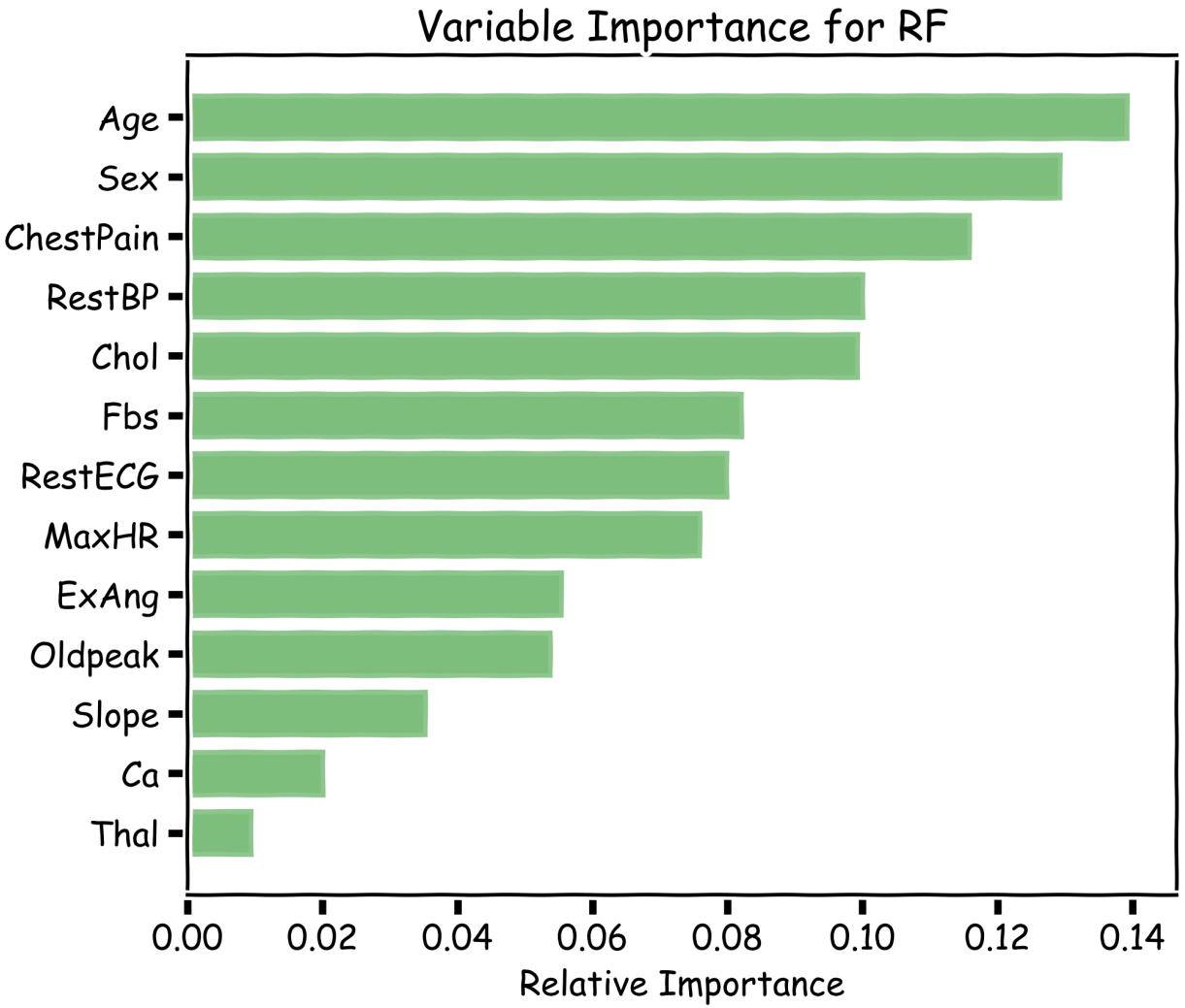
Tuning Random Forests

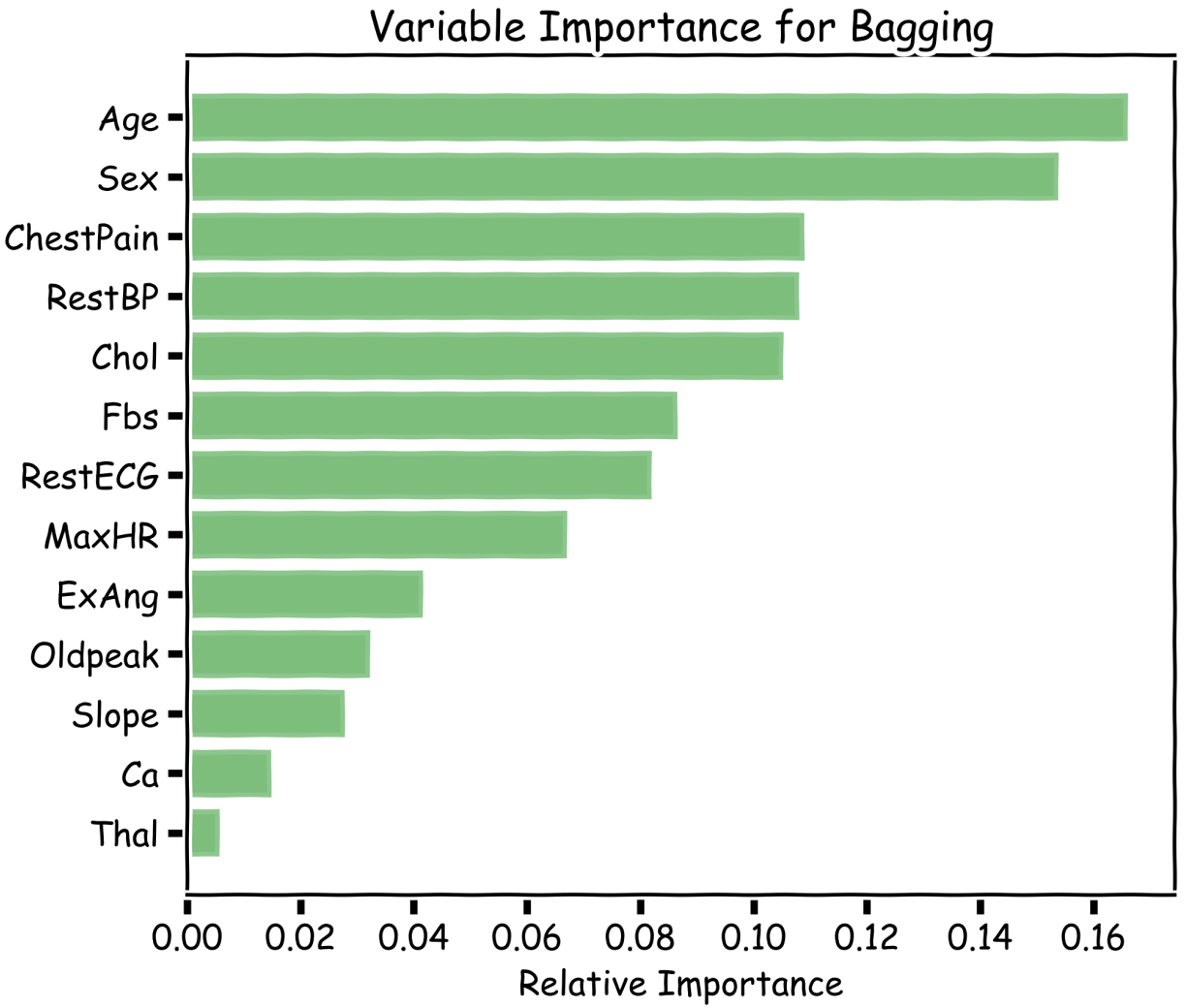
- There are standard (default) values for each of random forest hyper-parameters recommended by long time practitioners
- Generally, these parameters should be tuned through **OOB** (making them data-dependent and problem-dependent).
  - e.g. number of predictors to randomly select at each split:
    - $\sqrt{N_j}$ for classification
    - $\frac{N}{3}$ for regression
- Using out-of-bag errors, training and cross validation can be done in a single sequence - we cease training once the out-of-bag error stabilizes

Variable Importance for Random Forests

- Same as with Bagging:
  - Calculate the total amount that the RSE (for regression) or Gini index (for classification) is decreased due to splits over a given predictor, averaged over all B trees.
- **Alternative:**
  - Record the prediction accuracy on the *oob* samples for each tree.
  - Randomly permute the data for column j in the *oob* samples and record the accuracy again.
  - The decrease in accuracy as a result of this permuting is averaged over all trees and is used as a measure of the importance of variable j in the random forest.

Variable Importance for Random Forests





100 trees, max_depth=10

Final Thoughts on Random Forests

- When the number of predictors is large, but the **number of relevant predictors is small**, random forests can perform poorly.
  - In each split, the chances of selected a relevant predictor will be low and hence most trees in the ensemble will be weak models.

- Increasing the number of trees in the ensemble generally does not increase the risk of overfitting.

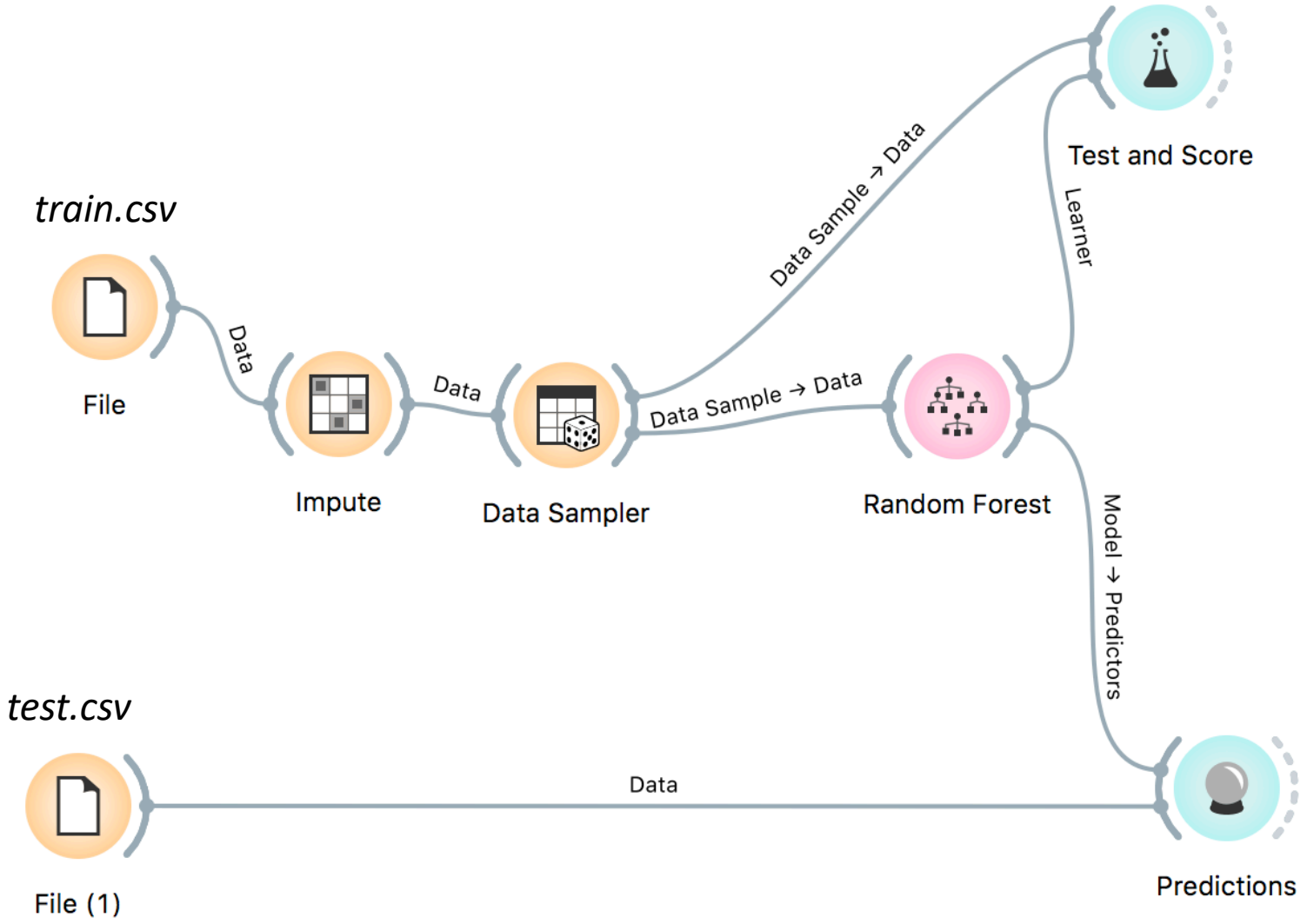
- Again, by decomposing the generalization error in terms of bias and variance, we see that increasing the number of trees produces a model that is at least as robust as a single tree.

  - However, if the number of trees is too large, then the trees in the ensemble may become more correlated, increase the variance.

Using Random Forest Classificator with Orange

- We can be using the [Kaggle Housing Prices] dataset to predict house prices using a Random Forest model.

- Download the *train.csv* and *test.csv* files from the kaggle site.



https://www.kaggle.com/c/house-prices-advanced-regression-techniques

Experimenting with the IRIS dataset

```
#Import scikit-learn dataset library
from sklearn import datasets
#Load dataset
iris = datasets.load_iris()

# Creating a DataFrame of given iris dataset.
import pandas as pd
data = pd.DataFrame({ 'sepal length':iris.data[:,0],
                      'sepal width':iris.data[:,1],
                      'petal length':iris.data[:,2],
                      'petal width':iris.data[:,3],
                      'species':iris.target })
```

Create train and test set with sklearn

```
# Import train_test_split function

from sklearn.model_selection import train_test_split
X = data[['sepal length', 'sepal width', 'petal length', 'petal width']] # Features
y = data['species'] # Labels

# Split dataset into training set and test set
# 70% training and 30% test

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3)
```

Building a RF model in sklearn

```
#Import Random Forest Model
from sklearn.ensemble import RandomForestClassifier

#Create a RF Classifier
clf = RandomForestClassifier(n_estimators = 100)

#Train the model using the training sets
clf.fit(X_train, y_train)

#Make the predictions
y_pred = clf.predict(X_test)
```

Testing the model in sklearn

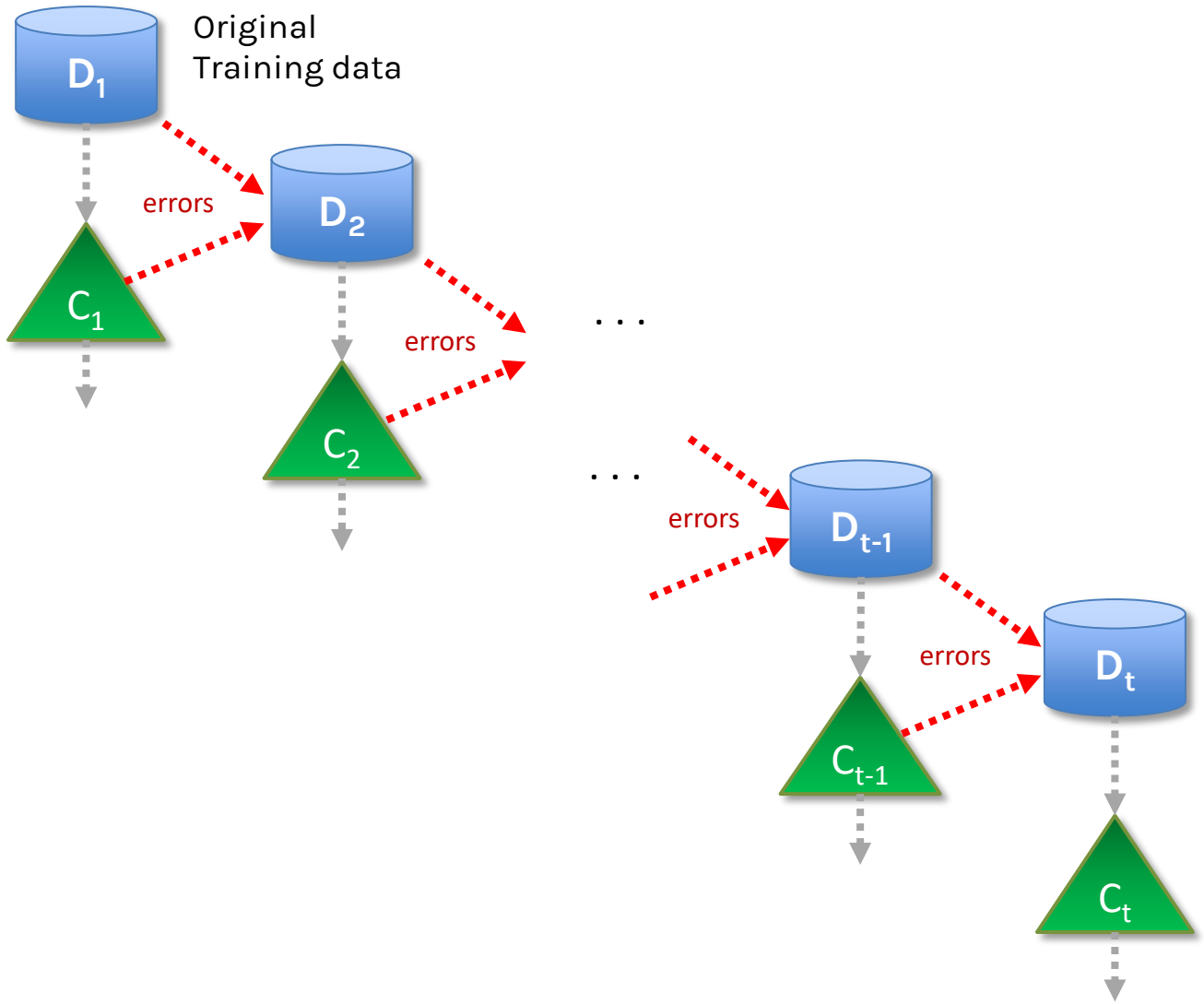
```
#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics

# Model Accuracy, how often is the classifier correct?
print("Accuracy: ", metrics.accuracy_score(y_test, y_pred))
```

```
Accuracy:  0.9333333333333335
```

Motivation for Boosting

- **Question:** Could we address the shortcomings of single decision trees models in some other way?
- For example, rather than performing variance reduction on complex trees, can we decrease the bias of simple trees - make them more expressive?
- Can we learn from our mistakes?
- A solution to this problem, making an expressive model from simple trees, is another class of ensemble methods called **boosting**.

General Idea (Boosting)



Boosting

- Sequential algorithm where at each step, a **weak learner** is trained based on the results of the previous learner.

- Two main types:
  - **Adaptive Boosting**: Reweight datapoints based on performance of last weak learner. Focuses on points where previous learner had trouble. Example: **AdaBoost**.
  - **Gradient Boosting**: Train new learner on residuals of overall model. Constitutes gradient boosting because approximating the residual and adding to the previous result is essentially a form of gradient descent. Example: **XGBoost**.

Gradient Boosting

Gradient Boosting

- The key intuition behind boosting is that one can take an **ensemble of simple models** $\{T_h\}_{h \in H}$ and **additively combine** them into a single, more complex model.

- Each model T_h might be a poor fit for the data, but a linear combination of the ensemble

$$T = \sum_h \lambda_h T_H$$

- can be expressive/flexible.

Gradient Boosting: the algorithm

- **Gradient boosting** is a method for iteratively building a complex regression model T by adding simple models. Each new simple model added to the ensemble compensates for the weaknesses of the current ensemble.

1. Fit a simple model $T^{(0)}$ on the training data

$$\{(x_1, y_1), \ldots, (x_N, y_N)\}$$

Set $T \leftarrow T^{(0)}$. Compute the residuals $\{r_1, \ldots, r_N\}$ for T.

2. Fit a simple model, $T^{(1)}$, to the current **residuals**, i.e. train using

$$\{(x_1, r_1), \ldots, (x_N, r_N)\}$$

3. Set $T \leftarrow T + \lambda T^{(1)}$

4. Compute residuals, set $r_n \leftarrow r_n - \lambda T^i(x_n), \ n = 1, \ldots, N$

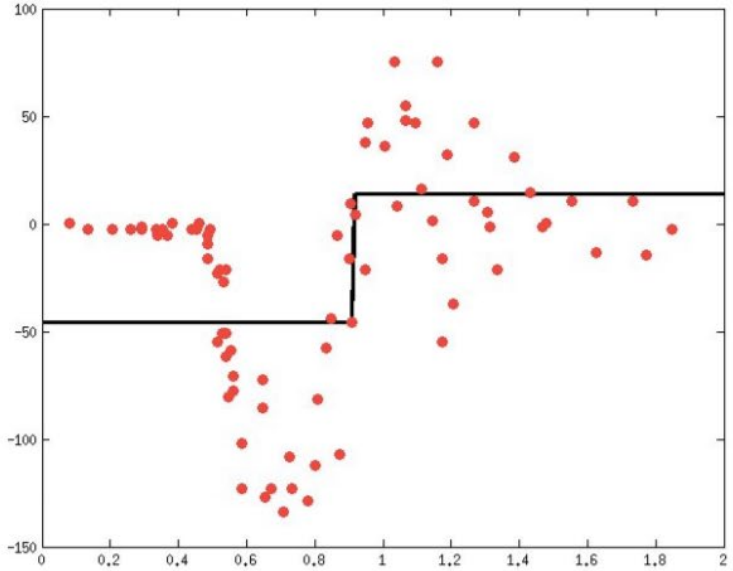
5. Repeat steps 2-4 until **stopping** condition met. Where $\boldsymbol{\lambda}$ is a constant called the **learning rate**.

Gradient Boosting (GB) vs Gradient Descent (GD)

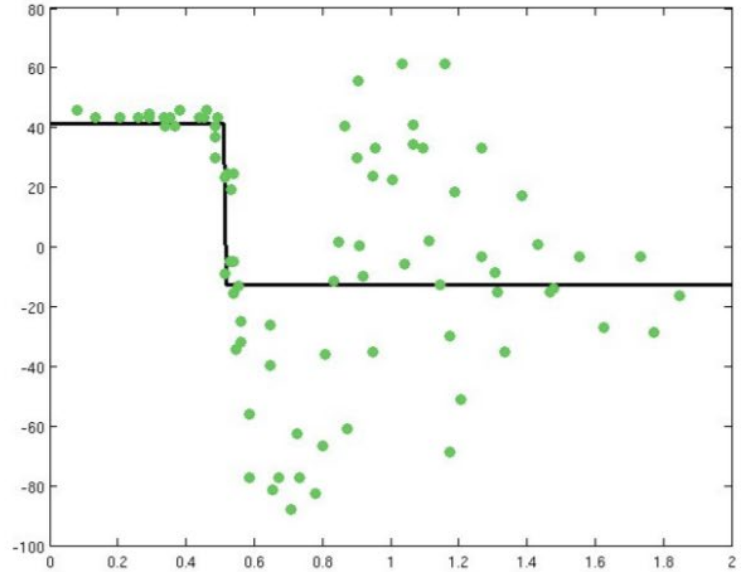
- Initialization:
  - GD: random/predefined initialization of weights/coefficients.
  - GB: train a weak model on training data.
- Error calculation:
  - GD: error/loss on entire training set.
  - GB: error/residuals of weak model on training set.
- Update/Fitting:
  - GD: update weights in opposite direction of gradient.
  - GB: fit new model to predict residual errors of previous model.
- Combination:
  - GD: no combination needed.
  - GB: combine predictions of all models to make final prediction.
- Convergence:
  - GD: repeat steps until convergence.
  - GB: repeat steps adding new models until stopping criteria met.

Gradient Boosting: illustration

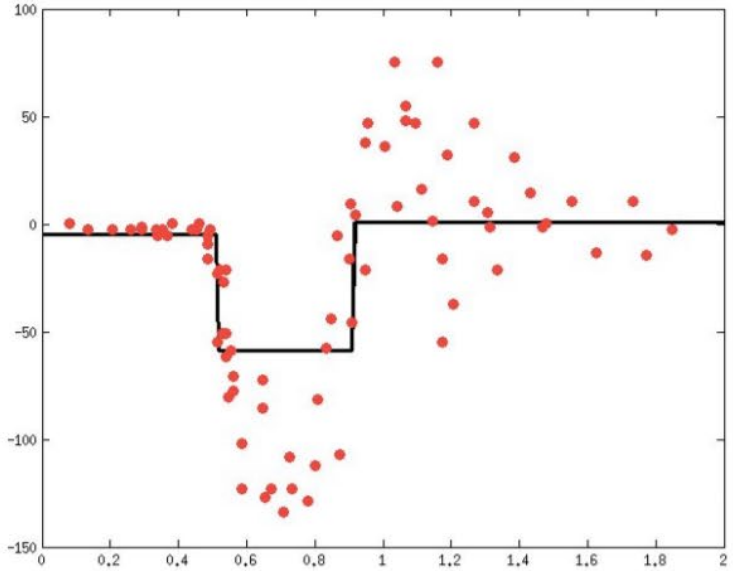
Learn a simple predictor…



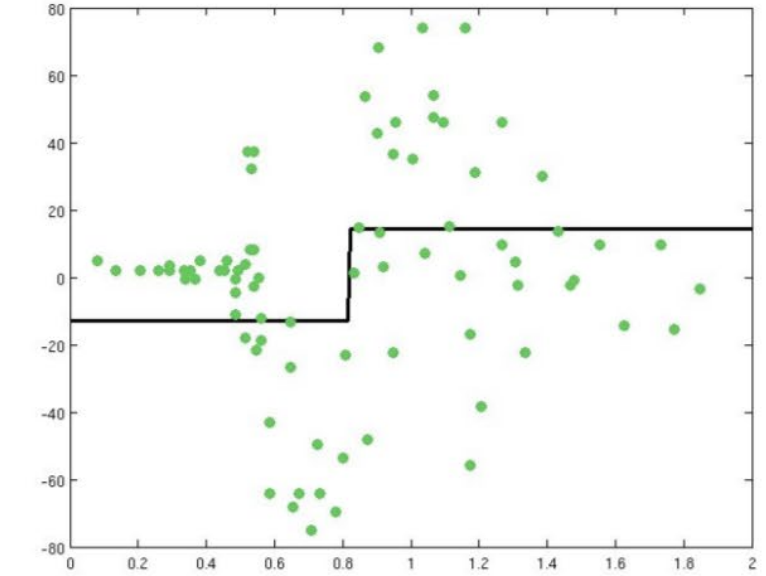
Then try to correct its errors



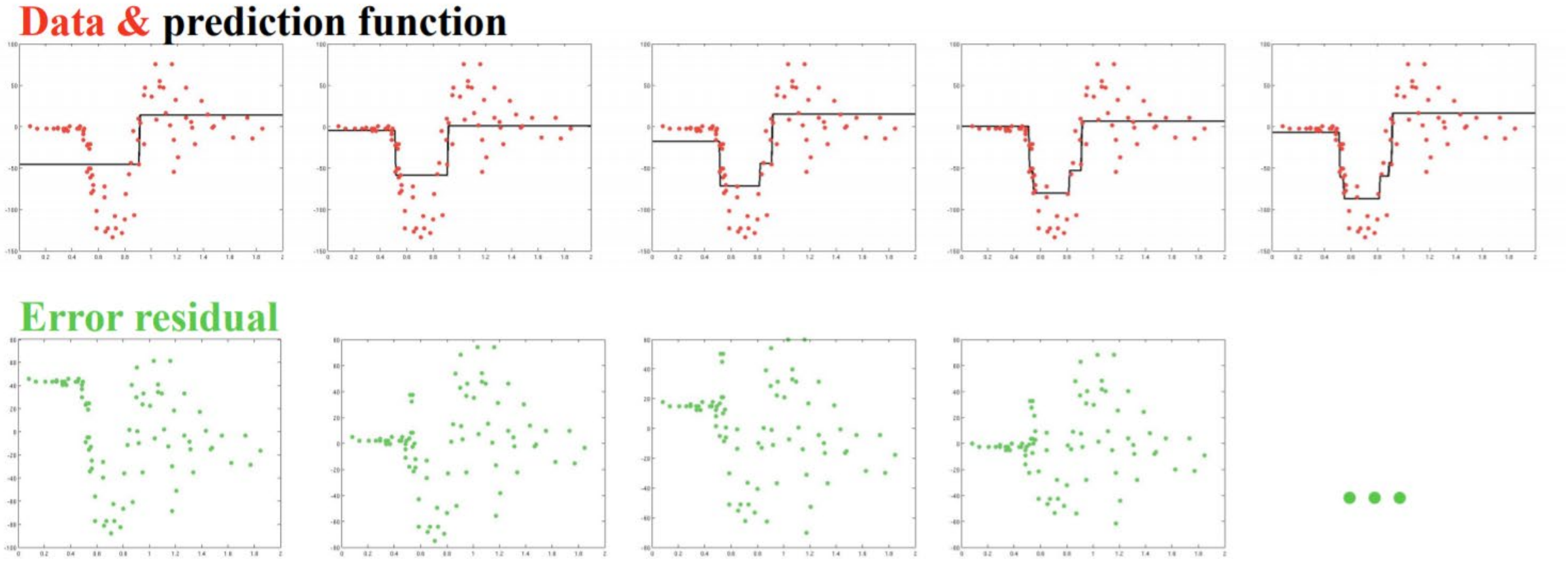
Combining gives a better predictor…



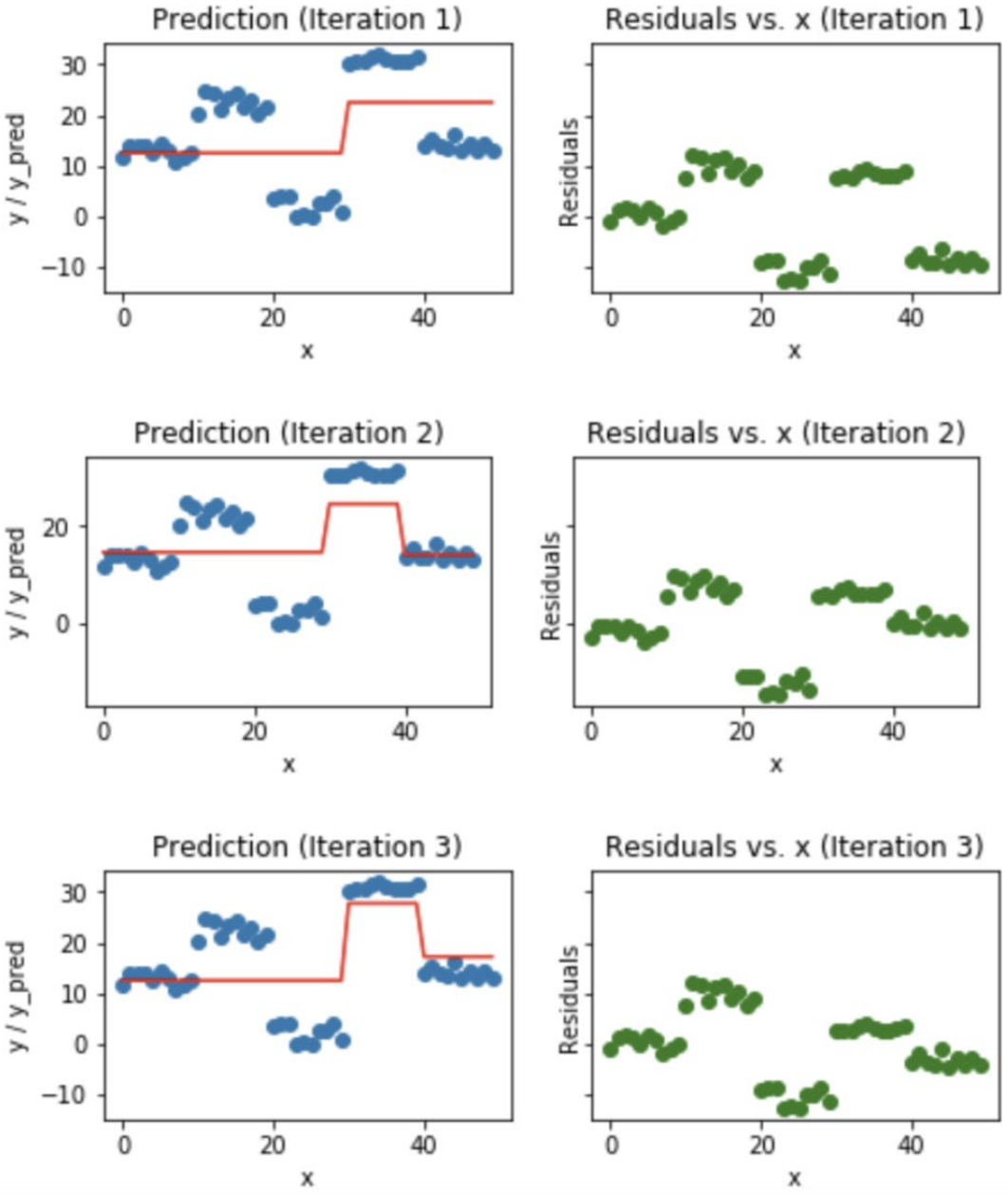
Can try to correct its errors also, & repeat

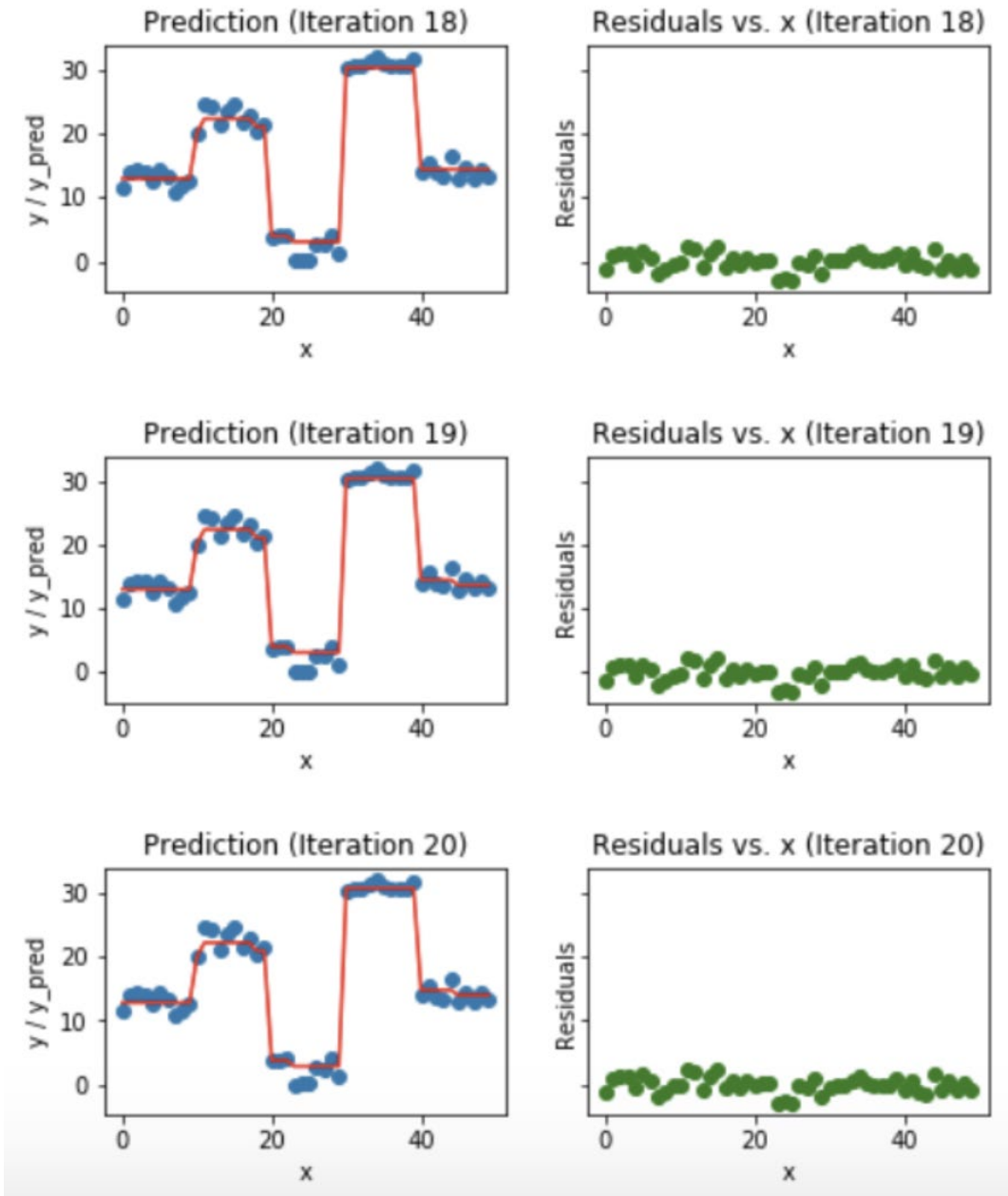


Gradient Boosting: illustration



Gradient Boosting: illustration 2





Why Does Gradient Boosting Work?

- Intuitively, each simple model $T^{(i)}$ we add to our ensemble model T, models the errors of T.
- Thus, with each addition of $T^{(i)}$, the residual is reduced:

$$r_n - \lambda T^{(i)}(x_n)$$

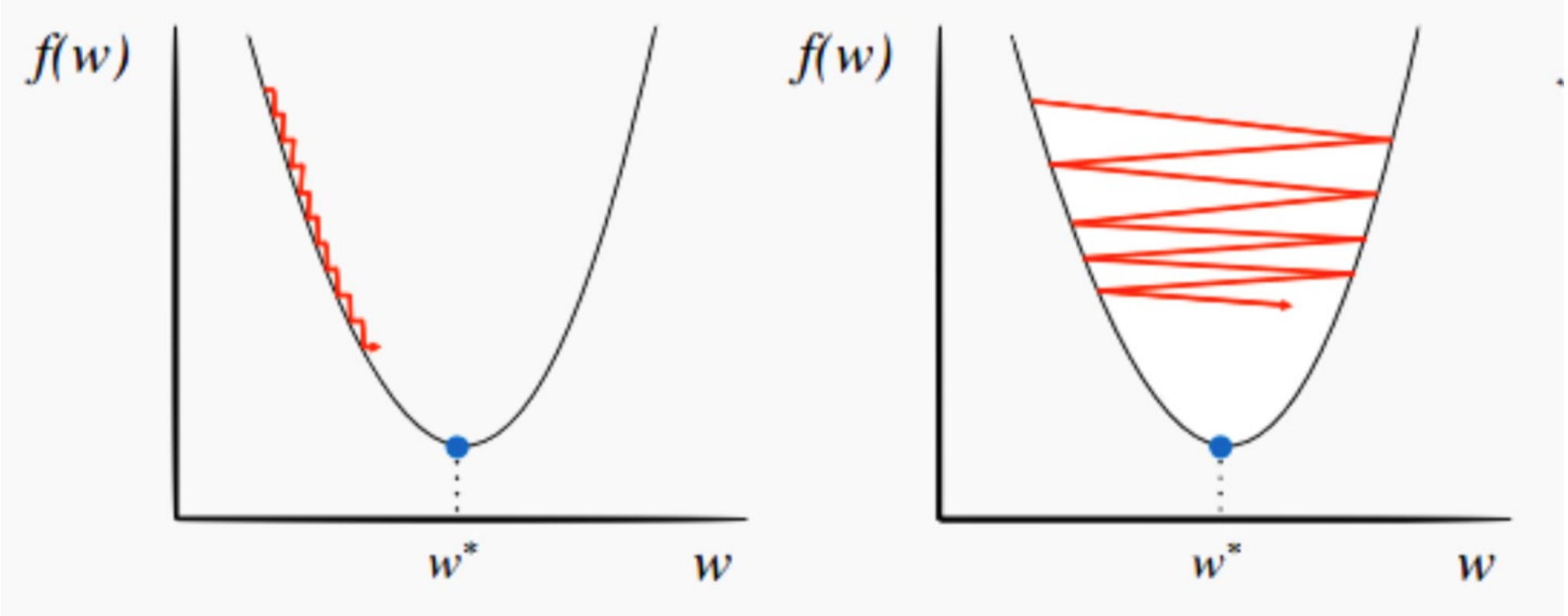
- **Note** that gradient boosting has a tuning parameter, λ.
  - If we want to easily reason about how to choose λ and investigate the effect of λ on model T, we need a bit more mathematical formalism.
  - How can we effectively descend through this optimization via an iterative algorithm?
  - We need to formulate gradient boosting as a type of **gradient descent**.

Choosing a Learning Rate

- Under ideal conditions, gradient descent iteratively approximates and converges to the optimum.
- **When do we terminate gradient descent?**
  - We can limit the number of iterations in the descent. But for an arbitrary choice of maximum iterations, we cannot guarantee that we are sufficiently close to the optimum in the end.
  - If the descent is stopped when the updates are sufficiently small (e.g. the residuals of T are small), we encounter a new problem: the algorithm may never terminate!
- **Both problems have to do with the magnitude of the learning rate, λ.**

Choosing a Learning Rate

- For a constant learning rate, λ, if $\boldsymbol{\lambda}$ **is too small**, it takes too many iterations to reach the optimum.
- If $\boldsymbol{\lambda}$ **is too large**, the algorithm may 'bounce' around the optimum and never get sufficiently close.



Choosing a Learning Rate

- Choosing λ:
  - If λ is a constant, then it should be tuned through cross validation.
  - For better results, use a variable λ. That is, let the value of λ depend on the gradient

$$\lambda = h(\|\nabla f(x)\|),$$

  - where $\|\nabla f(x)\|$ is the magnitude of the gradient, $\nabla f(x)$. So
    - around the optimum, when the gradient is small, λ should be small
    - far from the optimum, when the gradient is large, λ should be larger

Final thoughts on Boosting

- There are few implementations on boosting:
  - **XGBoost**: An efficient Gradient Boosting Decision
  - **LGBM**: Light Gradient Boosted Machines. It is a library for training GBMs developed by Microsoft, and it competes with XGBoost
  - **CatBoost**: A new library for Gradient Boosting Decision Trees, offering appropriate handling of categorical features

eXtreme Gradient Boosting

XGBoost

- XGBoost is essentially a very efficient Gradient Boosting Decision Tree implementation with some interesting features:
  - **Regularization:** Can use L1 or L2 regularization.
  - **Handling sparse data:** Incorporates a sparsity-aware split finding algorithm to handle different types of sparsity patterns in the data.
  - **Weighted quantile sketch:** Uses distributed weighted quantile sketch algorithm to effectively handle weighted data.
  - **Block structure for parallel learning:** Makes use of multiple cores on the CPU, possible because of a block structure in its system design. Block structure enables the data layout to be reused.
  - **Cache awareness:** Allocates internal buffers in each thread, where the gradient statistics can be stored.
  - **Out-of-core computing:** Optimizes the available disk space and maximizes its usage when handling huge datasets that do not fit into memory.

Bayes Classifier

- A probabilistic framework for solving classification problems
  - **A, C** random variables
  - **Joint** probability: **Pr(A=a, C=c)**
  - **Conditional** probability: **Pr(C=c | A=a)**
  - Relationship between joint and conditional probability distributions

$$\Pr(C, A) = \Pr(C|A) \times \Pr(A) = \Pr(A|C) \times \Pr(C)$$

- **Bayes Theorem:** $P(C|A) = \frac{P(A|C)P(C)}{P(A)}$

Bayesian Classifiers

- **Example**: How to classify the new record X = ('Yes', 'Single', 80K)

Tid	Refund	Matrial Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes
	A_1	A_2	A_3	C

Maximum Aposteriori Probability estimate:

- Find the value c for class C that maximizes P(C=c| X)

How do we estimate P(C|X) for the different values of C?

- We want to estimate P(C=Yes| X)
- and P(C=No| X)

Bayesian Classifiers

- Approach:
  - compute the posterior probability P(C | A_1, A_2, …, A_n) using the Bayes theorem:

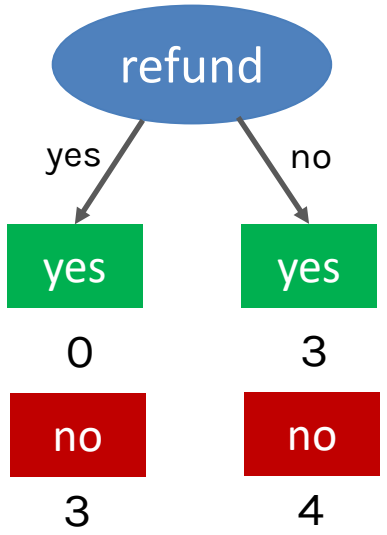
$$P(C|A_1, A_2, \ldots, A_n) = \frac{P(A_1 A_2 \ldots A_n|C)P(C)}{P(A_1 A_2 \ldots A_n)}$$

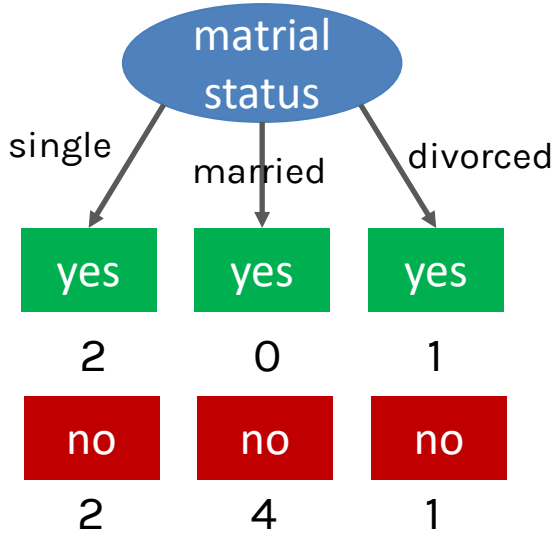
  - Maximizing P(C | A_1, A_2, …, A_n) is equivalent to maximizing P(A_1, A_2, …, A_n|C) P(C)
    - The value $P(A_1, \ldots, A_n)$ is the same for all values of C.

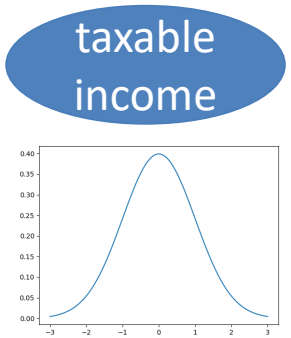
- How to estimate P(A_1, A_2, …, A_n | C)?
  - Assume **conditional independence** among attributes A_i when class C is given:
    - $P(A_1, A_2, \ldots, A_n|C) = P(A_1|C)\, P(A_2|C) \cdots P(A_n|C)$
    - **We can estimate $P(A_i|\, C)$ from the data.**

How to Estimate Probabilities from Data?

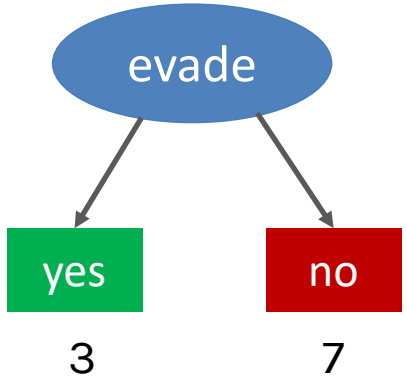
Tid	Refund	Matrial Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes





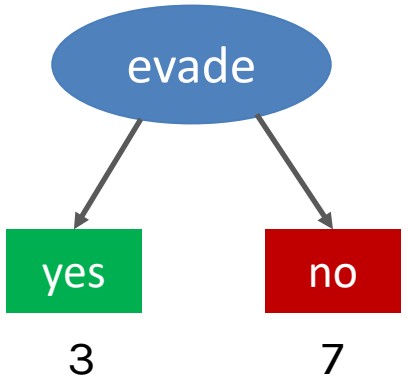


Gauss(104K, 1,874K)



How to Estimate Probabilities from Data?

Tid	Refund	Matrial Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes
	A_1	A_2	A_3	C



Class Prior Probability:

$$P(C = c) = \frac{N_c}{N}$$

N_c = Number of records with class c

N = Number of records

P(C = No) = 7/10

P(C = Yes) = 3/10

How to Estimate Probabilities from Data?

Tid	Refund	Matrial Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes
	A_1	A_2	A_3	C

Refund			Matrial Status			Evade	
	Yes	No		Yes	No	Yes	No
Yes	0	3	Single	2	2	3	7
No	3	4	Married	0	4		
			Divorced	1	1		

Discrete attributes:

Refund			Matrial Status			Evade	
	Yes	No		Yes	No	Yes	No
Yes	0/3	3/7	Single	2/3	2/7	3/10	7/10
No	3/3	4/7	Married	0/3	4/7		
			Divorced	1/3	1/7		

How to Estimate Probabilities from Data?

Tid	Refund	Matrial Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes
	A_1	A_2	A_3	C

Numerical values:

- Normal distribution:

$$P(A_i = a \mid c_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} e^{-\frac{(a-\mu_{ij})^2}{2\sigma_{ij}^2}}$$

  - One for each (a, c) pair

C = Yes

sample mean μ = 90
sample variance σ^2= 25

$$P(Income = 80|Yes) = \frac{1}{\sqrt{2\pi \cdot 25}} e^{-\frac{(80-90^2}{2\cdot 25}} = 0{,}01$$

C = No

sample mean μ = 110
sample variance σ^2= 2975

$$P(Income = 80|No) = \frac{1}{\sqrt{2\pi \cdot 2975}} e^{-\frac{(80-110^2}{2\cdot 2975}} = 0{,}0062$$

Bayesian Classifiers

$$P(C \mid A_1, A_2, \ldots, A_n) = P(A_1, A_2, \ldots, A_n \mid C)\, P(C)$$
$$= P(C) * P(A_1 \mid C) * P(A_2 \mid C) * \ldots * P(A_n \mid C)$$

- **Record**:
  X = (Refund='Yes', Status='Single', Income=80K)

Tid	Refund	Matrial Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes
	A_1	A_2	A_3	C

Refund			Matrial Status			Evade	
	Yes	No		Yes	No	Yes	No
Yes	0/3	3/7	Single	2/3	2/7	3/10	7/10
No	3/3	4/7	Married	0/3	4/7		
			Divorced	1/3	1/7		

We compute:

P(C = Yes|X) = P(C = Yes) *
P(Refund = Yes | C = Yes) *
P(Status = Single | C = Yes) *
P(Income = 80K | C = Yes) =
= 3/10* 0 * 2/3 * 0.01 = 0

P(C = No|X) = P(C = No) *
P(Refund = Yes | C = No) *
P(Status = Single | C = No) *
P(Income = 80K | C = No) =
= 7/10 * 3/7 * 2/7 * 0.0062 = 0.0005

Example of Naïve Bayes Classifier

- Creating a Naïve Bayes Classifier, essentially means to compute **counts**:

Total number of records: N = 10

Class No:
Number of records: 7
Attribute **Refund**:
Yes: 3
No: 4
Attribute **Marital Status**:
Single: 2
Divorced: 1
Married: 4
Attribute **Taxable Income**:
mean: 110
variance: 2975

Class Yes:
Number of records: 3
Attribute **Refund**:
Yes: 0
No: 3
Attribute **Marital Status**:
Single: 2
Divorced: 1
Married: 0
Attribute **Taxable Income**:
mean: 90
variance: 25

What happens when we get 0 for some of the counts?

Naïve Bayes Classifier

- If one of the conditional probability is **zero**, then the entire expression becomes zero

- **Laplace Smoothing**:

$$P(A_i = a|C = c) = \frac{N_{ac} + 1}{N_c + N_i}$$

  - N_i : number of attribute values for attribute A_i

Example of Naïve Bayes Classifier

Naïve Bayes Classifier

P(Refund=Yes|No) = 3/7
P(Refund=No|No) = 4/7
P(Refund=Yes|Yes) = 0
P(Refund=No|Yes) = 1

P(Status=Single|No) = 2/7
P(Status=Divorced|No) = 1/7
P(Status=Married|No) = 4/7
P(Status=Single|Yes) = 2/7
P(Status=Divorced|Yes) = 1/7
P(Status=Married|Yes) = 0

For taxable income:
If class=No:
sample mean=110
sample variance=2975
If class=Yes:
sample mean=90
sample variance=25

$P(X|Class=No) = P(Refund=No|Class=No) \times P(Married|\ Class=No) \times P(Income=120K|\ Class=No)$
$= 3/7 \times 2/7 \times 0.0062$
$= 0.00075$

$P(X|Class=Yes) = P(Refund=No|\ Class=Yes) \times P(Married|\ Class=Yes) \times P(Income=120K|\ Class=Yes)$
$= 0 \times 2/3 \times 0.01$
$= 0$

$P(No) = 0.7,\ P(Yes) = 0.3$
$P(X|No) \times P(No) = 0.000525$
$P(X|Yes) \times P(Yes) = 0$

Naïve Bayes Classifier with Laplacian Smoothing

P(Refund=Yes|No) = 4/9
P(Refund=No|No) = 5/9
P(Refund=Yes|Yes) = 1/5
P(Refund=No|Yes) = 4/5

P(Status=Single|No) = 3/10
P(Status=Divorced|No) = 2/10
P(Status=Married|No) = 5/10
P(Status=Single|Yes) = 3/6
P(Status=Divorced|Yes) = 2/6
P(Status=Married|Yes) = 1/6

For taxable income:
If class=No:
sample mean=110
sample variance=2975
If class=Yes:
sample mean=90
sample variance=25

$P(X|Class=No) = P(Refund=No|Class=No) \times P(Married|\ Class=No) \times P(Income=120K|\ Class=No)$
$= 4/9 \times 3/10 \times 0.0062$
$= 0.00082$

$P(X|Class=Yes) = P(Refund=No|\ Class=Yes) \times P(Married|\ Class=Yes) \times P(Income=120K|\ Class=Yes)$
$= 1/5 \times 3/6 \times 0.01$
$= 0.001$

$P(No) = 0.7,\ P(Yes) = 0.3$
$P(X|No) \times P(No) = 0.0005$
$P(X|Yes) \times P(Yes) = 0.0003$

Implementation details

- Computing the conditional probabilities involves multiplication of many very small numbers
  - Numbers get very close to zero, and there is a danger of numeric instability

- We can deal with this by computing the **logarithm** of the conditional probability

$$\log P(C|A) \sim \log P(A|C) + \log P(C)$$
$$= \sum_i \log P(A_i|C) + \log P(C)$$

Naïve Bayes for Text Classification

- Naïve Bayes is commonly used for text classification
- For a document with k terms $d = (t_1, \dots, t_k)$

$$P(c|d) = P(c)P(d|c) = P(c)\prod_{t_i \in d} P(t_i|c)$$

Fraction of documents in c

- $P(t_i|c)$= Fraction of terms from all documents in c that are t_i.

$$\boldsymbol{P(t_i|c) = \frac{N_{ic} + 1}{N_c + T}}$$

Number of times t_i appears in some document in c

Laplace Smoothing

Total number of terms in all documents in c

Number of unique words (vocabulary size)

- Easy to implement and works relatively well
- Limitation: Hard to incorporate additional features (beyond words).
  - E.g., number of adjectives used.

Multinomial document model

- Probability of document $d = (t_1, \ldots, t_k)$ in class c:

$$P(c|d) = P(c) \prod_{t_i \in d} P(t_i|c)$$

Example

News titles for Politics and Sports

	Politics	Sports
documents	"Obama meets Merkel" "Obama elected again" "Merkel visits Greece again"	"OSFP European basketball champion" "Miami NBA basketball champion" "Greece basketball coach?"
	P(p) = 0.5	P(s) = 0.5
terms Vocabulary size: 14	obama:2, meets:1, merkel:2, elected:1, again:2, visits:1, greece:1	OSFP:1, european:1, basketball:3, champion:2, miami:1, nba:1, greece:1, coach:1
	Total terms: 10	Total terms: 11

New title: X = "Obama likes basketball"

P(Politics|X) ~ P(p)*P(obama|p)*P(likes|p)*P(basketball|p)
= 0.5 * 3/(10+14) *1/(10+14) * 1/(10+14) = 0.000108

P(Sports|X) ~ P(s)*P(obama|s)*P(likes|s)*P(basketball|s)
= 0.5 * 1/(11+14) *1/(11+14) * 4/(11+14) = 0.000128

Naïve Bayes (Summary)

- Robust to isolated noise points
- Handle missing values by ignoring the instance during probability estimate calculations
- Robust to irrelevant attributes
- Independence assumption may not hold for some attributes
  - Use other techniques such as Bayesian Belief Networks (BBN)
- Naïve Bayes can produce a probability estimate, but it is usually a very biased one
  - Logistic Regression is better for obtaining probabilities.

Using Naïve Bayes Classifier with sklearn

```
#Import Gaussian Naive Bayes model
from sklearn.naive_bayes import GaussianNB

#Create a Gaussian Classifier
gnb = GaussianNB()

#Train the model using the training sets
gnb.fit(X_train, y_train)

#Predict the response for test dataset
y_pred = gnb.predict(X_test)

#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics

# Model Accuracy, how often is the classifier correct? print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

Accuracy: 0.9074074074074744)